AI FOR FINANCE – CIA-2

AI-Powered Project Development

FraudShield AI

AI-Powered Fraud Detection System

Step-by-Step Project Report

Project Option 3 – flags suspicious financial transactions based on unusual patterns

---

Student name : _____

Register number : _____

Class / Section : _____

Faculty : _____

Date of submission : _____

Contents

Part A – Project at a Glance 3
A1. Objective 3
A2. The problem 3
A3. The solution in one picture 3
A4. Key results 3
A5. Assignment requirements checklist 3
Part B – Step-by-Step Development 5
Step 1 – Understand the problem and choose the project 5
Step 2 – Choose the tools and set up the environment 5
Step 3 – Create the transaction dataset 5
Step 4 – Feature engineering 7
Step 5 – Train AI model 1: Random Forest (supervised learning) 7
Step 6 – Train AI model 2: Isolation Forest (anomaly detection) 8
Step 7 – Combine into a hybrid score and tune the threshold 8
Step 8 – Make the AI explainable 8
Step 9 – Evaluate the model on unseen data 9
Step 10 – Build the web application 10
Step 11 – Test the application with sample data 11
Step 12 – Package and submit 15
Part C – Step-by-Step User Guide 17
C1. Start the application 17
C2. Check a single transaction 17
C3. Scan a batch of transactions 17
C4. Other pages 17
Part D – Applications, Limitations and Conclusion 19
D1. Benefits and practical applications 19
D2. Limitations 19
D3. Future work 19
D4. Conclusion 19
Appendix A – How AI tools were used to build the project 19

Part A – Project at a Glance

A1. Objective

Build a working application that uses Artificial Intelligence to analyse financial transactions and **identify potentially fraudulent activity based on unusual transaction patterns** – and explain each decision in plain language.

A2. The problem

- **Fraud is rare** – about 3% of transactions – so a model that always says "genuine" is 97% accurate and still useless.
- **Fraud keeps changing** – fixed rules such as "block everything above ₹50,000" quickly become outdated.
- **Mistakes are costly both ways** – a missed fraud loses money; a false alarm blocks a genuine customer.

A3. The solution in one picture

Stage	What happens
1. Input	Transaction details – amount, customer's usual spend, merchant, channel, time, distance, device, recent activity
2. Feature engineering	Raw fields become behavioural signals (e.g. amount ÷ usual spend, night-time flag)
3. AI model 1 – Random Forest	Supervised learning: probability that the transaction is fraud
4. AI model 2 – Isolation Forest	Anomaly detection: how unusual the transaction is
5. Hybrid risk score	0.75 × Random Forest + 0.25 × Isolation Forest → score 0–100
6. Decision	Low → approve · Medium → verify with OTP · High → block
7. Explanation	Plain-language red flags + Shapley-value chart

A4. Key results

Measure	Result
Fraud caught (recall)	87.7%
Alerts that were real fraud (precision)	90.6%
Fraud money caught	96.5%
Genuine customers wrongly flagged	0.3%
ROC AUC (1.0 = perfect)	0.998

A5. Assignment requirements checklist

General requirement	Where it is met
Use AI tools to assist in designing and developing the project	Step 2 and Appendix A – an AI coding assistant (Claude Code) was used to plan, code, debug and test
Build a working application with a user-friendly interface	Step 10 – Streamlit web app with 5 pages, one-click examples and colour-coded results
Demonstrate the main features with sample data	Step 11 – three example transactions, a 10-customer Excel file and a 1,000-transaction batch

General requirement	Where it is met
Explain how AI is used in the project	Steps 5–8 and the app's "How the AI Works" page
Submit the project source code and a brief project report	Step 12 – GitHub repository, submission zip and this report
Present the project and explain practical applications	Part C (user guide) and Part D (applications)

Part B – Step-by-Step Development

The project was built in 12 steps. Each step lists what was done, why, and its output.

Step 1 – Understand the problem and choose the project

1. Read the CIA-2 assignment brief and its five project options.
2. Chose **Option 3 – AI-Powered Fraud Detection System**: build a system that analyses financial transactions and flags fraud based on unusual patterns.
3. Studied how real card fraud behaves: stolen cards are used quickly, at odd hours, far from home, for big amounts at merchants where goods are easy to resell (gift cards, crypto, electronics).
4. Defined success: catch most fraud **and** keep false alarms on genuine customers low, and **explain** every decision.

> **✓ Output of this step**
> Clear problem statement and list of fraud warning signs to model.

Step 2 – Choose the tools and set up the environment

Tool	Used for
Python 3.10	Programming language
pandas, NumPy	Data handling and calculations
scikit-learn	AI models – Random Forest and Isolation Forest; evaluation metrics
Streamlit	Web application (user interface) written in pure Python
Plotly	Interactive charts inside the app
joblib	Saving and loading the trained model
openpyxl	Reading Excel uploads
Claude Code (AI coding assistant)	Designing the architecture, writing and debugging code, testing the app in a browser
Git and GitHub	Version control and code submission

Installation command:

```
pip install streamlit scikit-learn pandas numpy plotly joblib openpyxl
```

> **✓ Output of this step**
> A working Python environment with all libraries installed.

Step 3 – Create the transaction dataset

Real bank data is confidential, so a realistic synthetic dataset was generated by `data_generator.py`.

1. Created **600 customers**, each with a home city and a typical spend per transaction (most around ₹1,500, a few big spenders).

2. Generated **24,250 genuine transactions** over 90 days (July–September 2026) – mostly daytime, close to home, at normal amounts. A few genuine ones are deliberately unusual (travel, occasional big purchases) to make the problem realistic.
3. Injected **750 fraud transactions (3%)** in short bursts, using real fraud patterns: amounts 3–15× the usual spend, night hours, far away or abroad, new devices, high-risk merchants. Not every fraud shows every signal.
4. Calculated history-based fields for every transaction: the customer's usual spend so far and the number of transactions in the last 24 hours.
5. Saved `data/transactions.csv` (25,000 labelled rows) and `data/sample_new_transactions.csv` (1,000 new unlabelled rows for testing).

Field	Meaning
amount	Transaction amount (₹)
customer_avg_amount	Customer's usual transaction amount (₹)
merchant_category	Groceries, Restaurants, Fuel, Utilities, Clothing, Travel, Electronics, Online Shopping, Gift Cards, Crypto Exchange
channel	POS (card swipe), Online, ATM
hour	Hour of day, 0–23
distance_from_home_km	Distance from the customer's home city
is_foreign	1 = international
is_new_device	1 = device never seen before
txn_count_24h	Customer's transactions in the last 24 hours
is_fraud	Label – 1 = confirmed fraud (used for training and testing only)

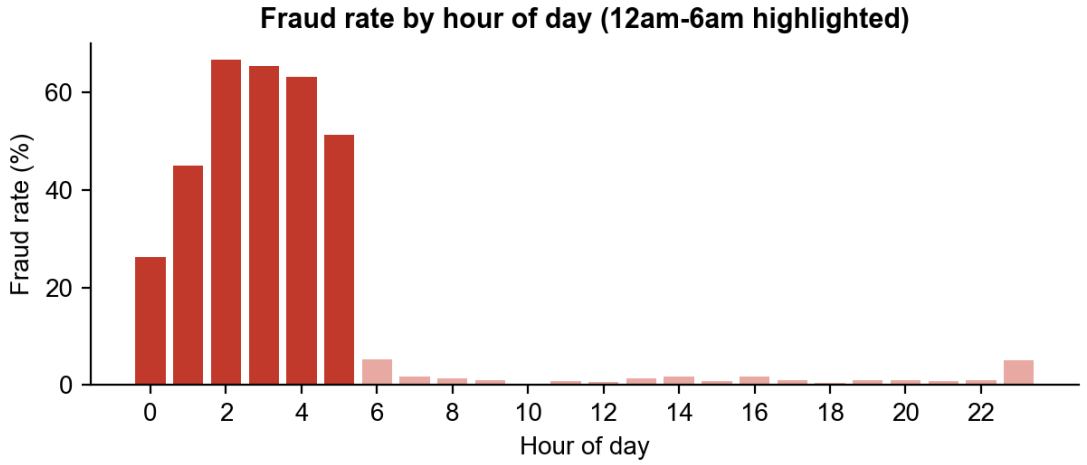


Figure 1 – Fraud rate by hour: fraud is concentrated between midnight and 6 am

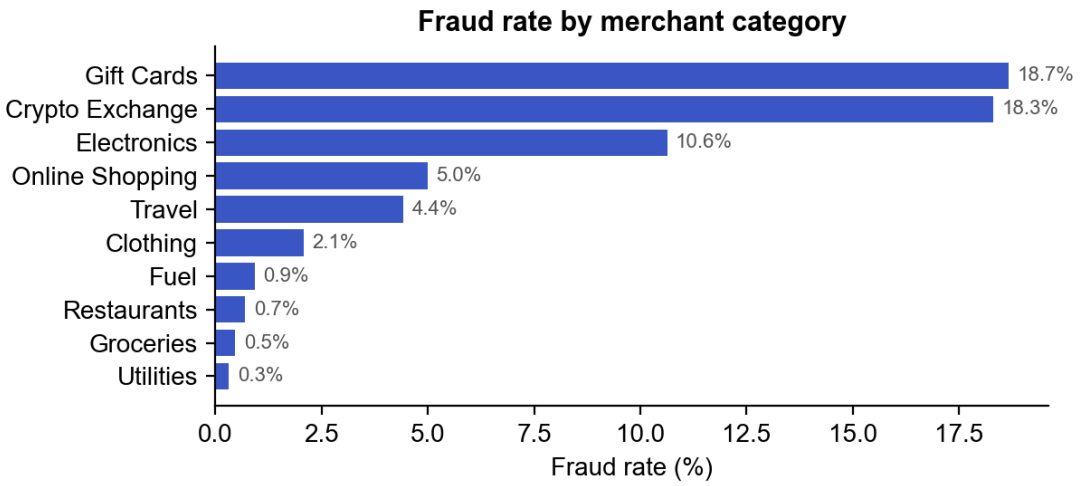


Figure 2 – Fraud rate by merchant category

> **✓ Output of this step**
> 25,000 labelled transactions (750 fraud) + 1,000 new transactions for testing.

Step 4 – Feature engineering

AI models learn better from **behavioural signals** than from raw numbers. The key idea: compare each transaction with **that customer's own normal behaviour**, not with one fixed limit for everybody.

Engineered feature	How it is calculated	Why it helps
amount_to_avg_ratio	amount ÷ customer_avg_amount	₹20,000 is normal for one customer and very suspicious for another
is_night	1 if hour < 6	Most fraud happens while the card owner is asleep
distance_from_home_km	From transaction location	Stolen card details are often used in another city or country
txn_count_24h	Count of the customer's transactions in 24 h	Fraudsters spend quickly before the card is blocked
merchant_category, channel	One-hot encoded (one column per value)	Some merchants and channels are much riskier

> **✓ Output of this step**
> 10 input features per transaction, prepared by `add_features()` in `fraud_model.py`.

Step 5 – Train AI model 1: Random Forest (supervised learning)

1. Split the data: **75% for training (18,750)** and **25% for testing (6,250)**, keeping the 3% fraud share in both (stratified split).
2. Inside the training part, kept 20% aside as a **validation set** for tuning (Step 7).
3. Trained a **Random Forest of 300 decision trees** (max depth 14). Each tree learns rules such as "amount > 4× usual AND hour < 6 → fraud"; the trees vote and the share of fraud votes is the probability.

4. Handled the rare fraud class with **class_weight = balanced_subsample**, so mistakes on fraud cases count more during training.

> **✓ Output of this step**
> A model that outputs a fraud probability (0–100%) for any transaction.

Step 6 – Train AI model 2: Isolation Forest (anomaly detection)

1. Trained an **Isolation Forest (300 trees)** on **genuine transactions only** – it learns what normal behaviour looks like.
2. It splits the data randomly; unusual transactions get separated ("isolated") in very few splits and receive a high anomaly score.
3. Scaled the anomaly score to 0–1 so it can be combined with the Random Forest probability.

Why a second model? The Random Forest only knows fraud patterns it has seen. The Isolation Forest flags anything unusual, so it can catch **new** fraud patterns too.

> **✓ Output of this step**
> A model that outputs an anomaly score (0–100%) for any transaction.

Step 7 – Combine into a hybrid score and tune the threshold

```
risk_score = 0.75 × Random Forest probability + 0.25 × Isolation Forest anomaly score
```

1. Calculated the hybrid risk score for every validation transaction.
2. Tried every possible alert threshold and picked the one with the best **F1 score** (balance of catching fraud vs. false alarms): **52.4 out of 100**.
3. Defined three risk levels and the action for each:

Risk level	Risk score	Action
Low	below 26.2	Approve automatically
Medium	26.2 – 52.4	Step-up verification: OTP or call to the customer
High	52.4 and above	Block the transaction and alert the fraud team

> **✓ Output of this step**
> A single 0–100 risk score with a clear decision for every transaction.

Step 8 – Make the AI explainable

Banks must be able to explain why a payment was blocked. Two explanation methods were added:

1. **Red-flag reasons** – simple checks that produce plain sentences, e.g. "Amount is 28.0× the customer's usual spend", "Unusual time (02:00)", "4,200 km from home", "New / unrecognised device".
2. **Shapley values** (cooperative game theory, the idea behind SHAP) – the transaction is compared with a typical genuine transaction of the same customer. All $2^8 = 256$

combinations of the 8 factors are scored, and the change in fraud probability is shared fairly between the factors. The result shows exactly how many percentage points each factor added or removed.

> **✓ Output of this step**
> Every decision comes with reasons and an explanation chart.

Step 9 – Evaluate the model on unseen data

All results are on the **6,250 test transactions the models never saw** during training or tuning.

Model	Precision	Recall	F1 score	ROC AUC
Random Forest alone	0.901	0.872	0.886	0.998
Isolation Forest alone	0.925	0.797	0.856	0.995
Hybrid (final system)	**0.906**	**0.877**	**0.891**	**0.998**

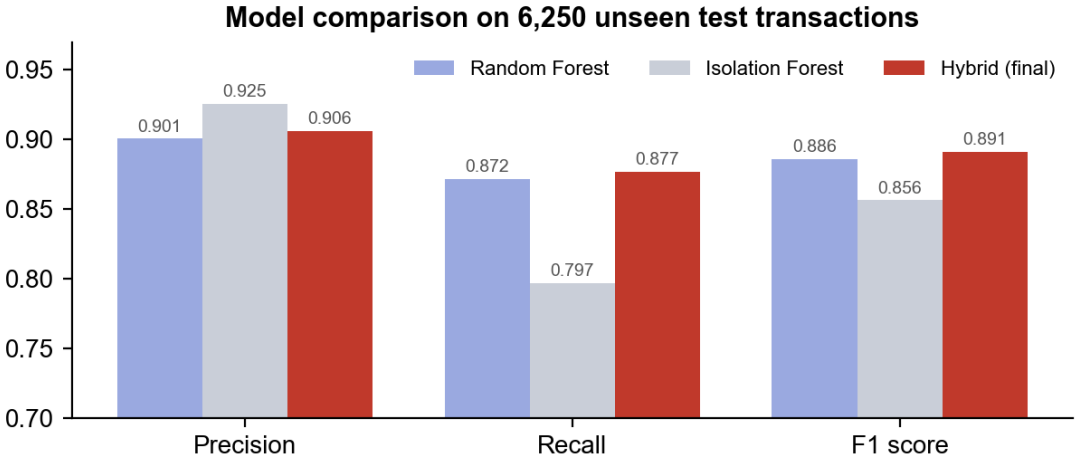


Figure 3 – The hybrid model has the best F1 score

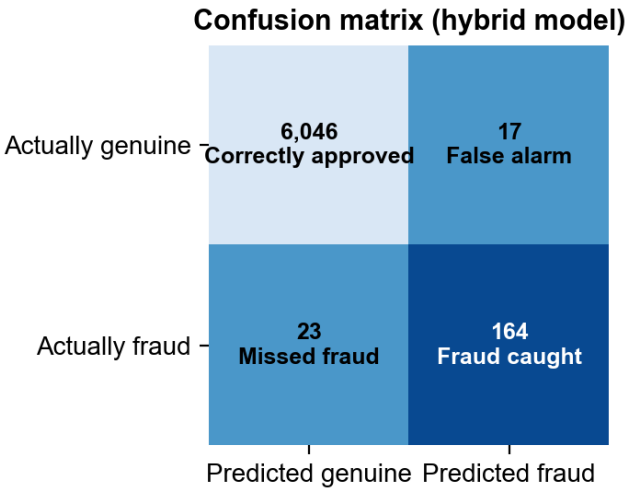


Figure 4 – Confusion matrix of the hybrid model

- **Precision 90.6%** – of all alerts, how many were real fraud.

- **Recall 87.7%** – of all real frauds, how many were caught (164 caught, 23 missed).
- **False alarms** – only 17 of 6,063 genuine transactions (0.3%).
- **96.5% of the fraud money** was caught – the large frauds are the easiest to detect.

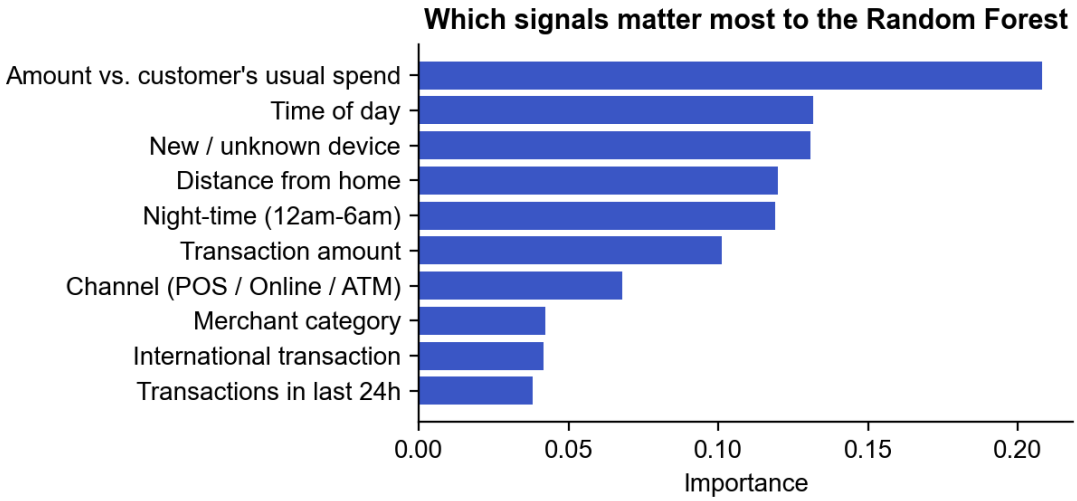


Figure 5 – Most important signals: amount vs. usual spend, time of day, new device

> **✓ Output of this step**
> Trained model saved to `models/fraud_model.joblib`, metrics to `models/metrics.json`.

Step 10 – Build the web application

The user interface was built with **Streamlit** in `app.py`. It has five pages, selected from the sidebar:

Page	Purpose
📊 Dashboard	KPIs and charts of the fraud patterns the AI learned from
🔍 Check a Transaction	Score one transaction in real time with a gauge, decision, red flags and Shapley chart
📁 Batch Scan (CSV / Excel)	Upload a file of transactions; get a ranked list of suspicious ones and download the results
📈 Model Performance	Confusion matrix, ROC curve, model comparison and feature importance
🤖 How the AI Works	Plain-language explanation of the approach and real-world uses

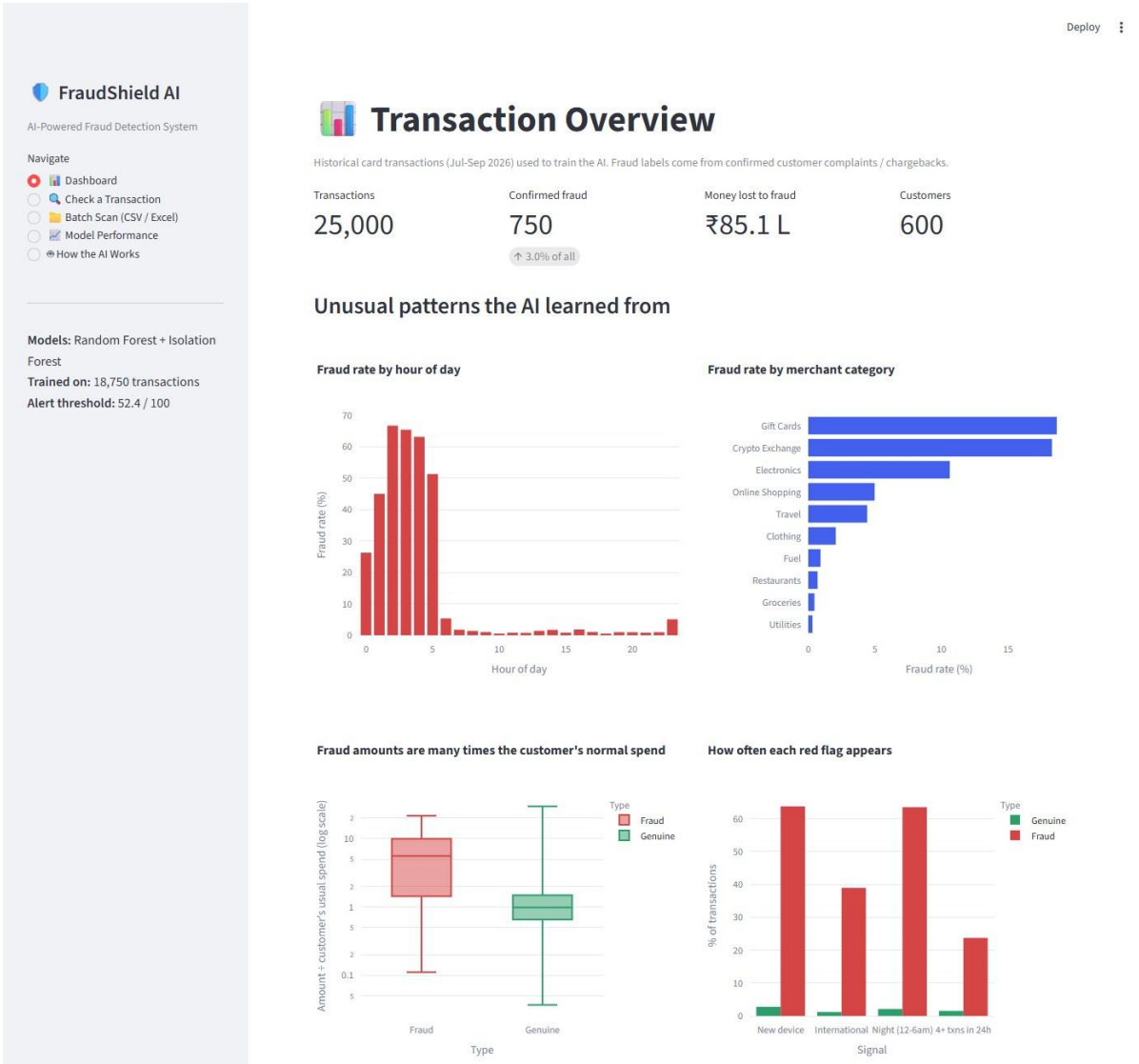


Figure 6 – Dashboard page

> **✓ Output of this step**
> A working, user-friendly web application: `python -m streamlit run app.py`.

Step 11 – Test the application with sample data

Test 1 – Three example transactions (Check a Transaction page)

Example	Key details	Expected	Result
Normal	₹1,200 groceries, 6 pm, 4 km from home, known device	Low	0.4 / 100 – Approve ✓
Borderline	₹6,500 travel (usual ₹1,800), 11 pm, 900 km away	Medium	34.7 / 100 – Verify ✓
Suspicious	₹42,000 electronics (usual ₹1,500), 2 am, abroad, new device, 5 txns in 24 h	High	100 / 100 – Block ✓

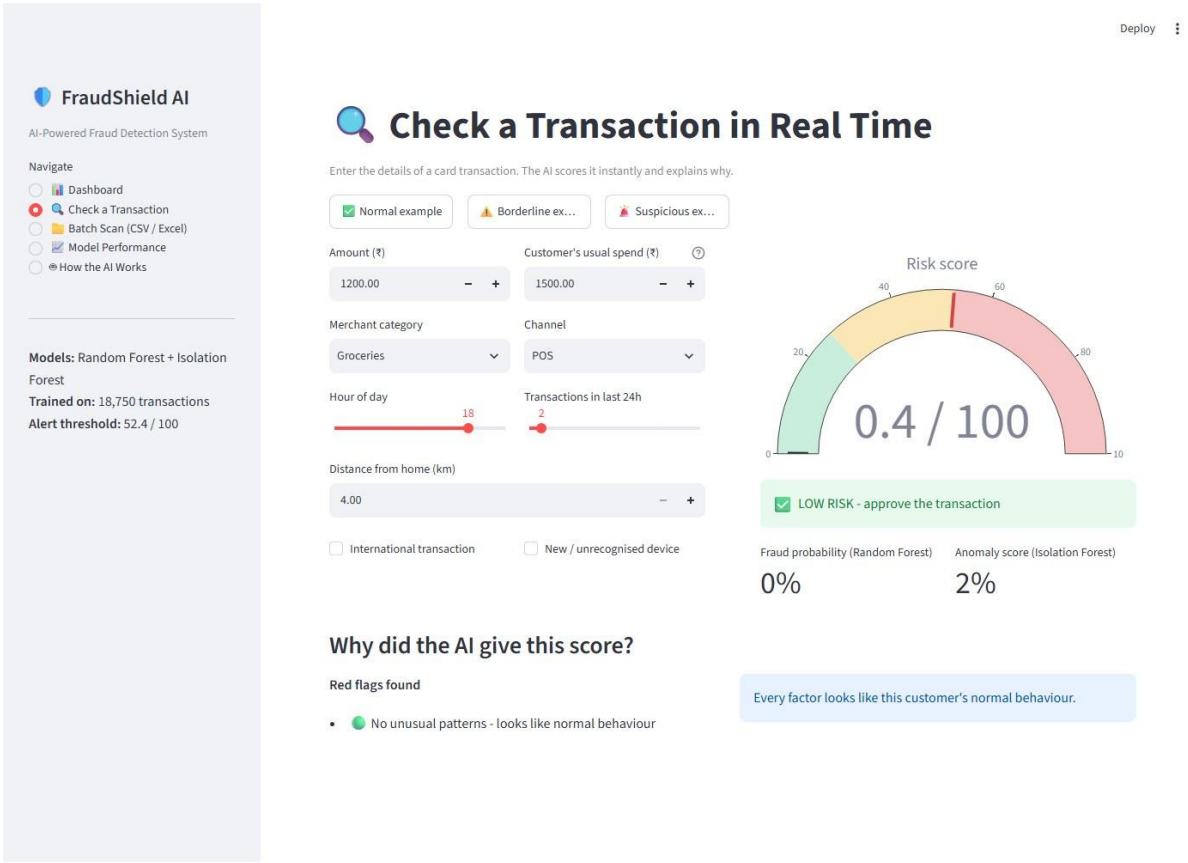


Figure 7 – Normal transaction: low risk, approve

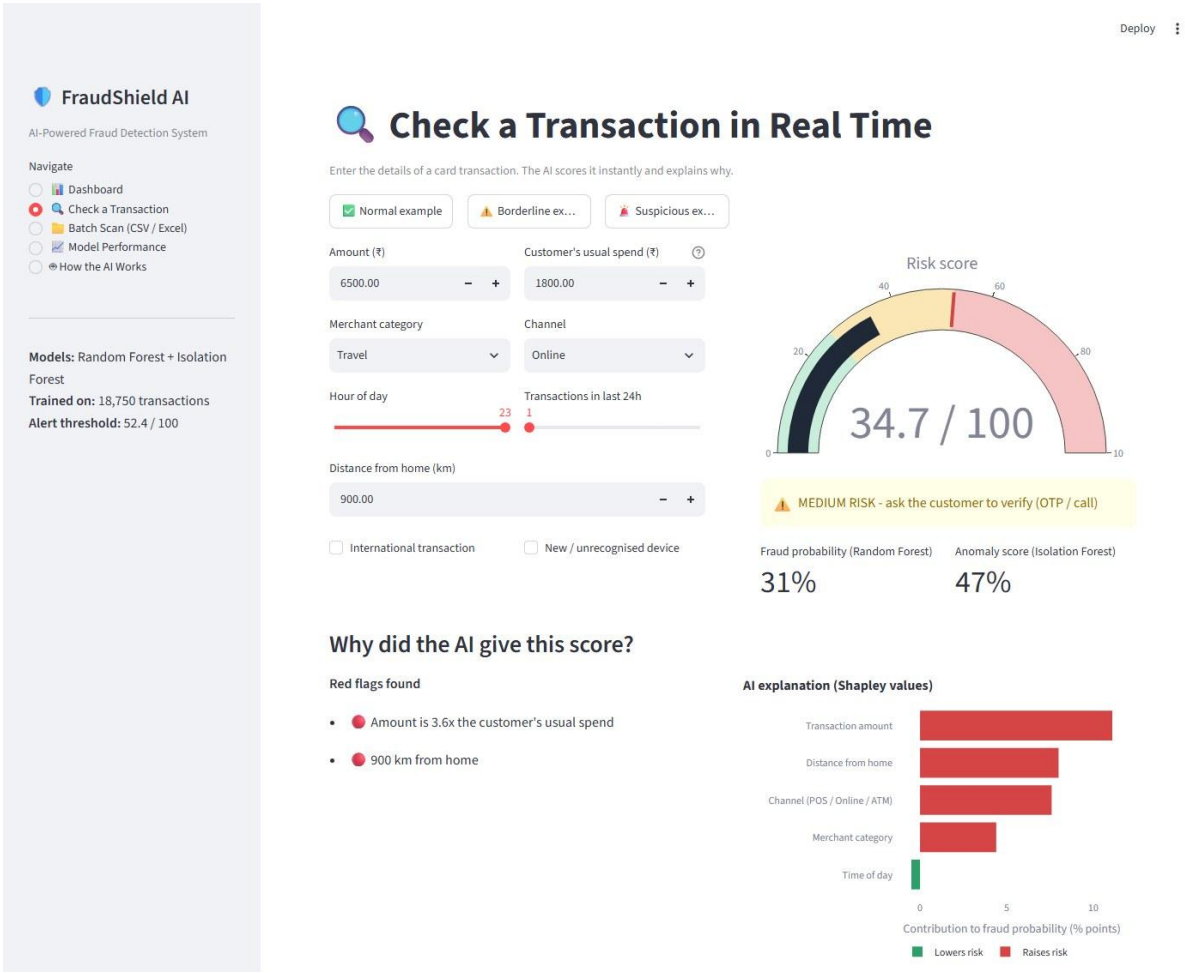


Figure 8 – Borderline transaction: medium risk, verify with OTP

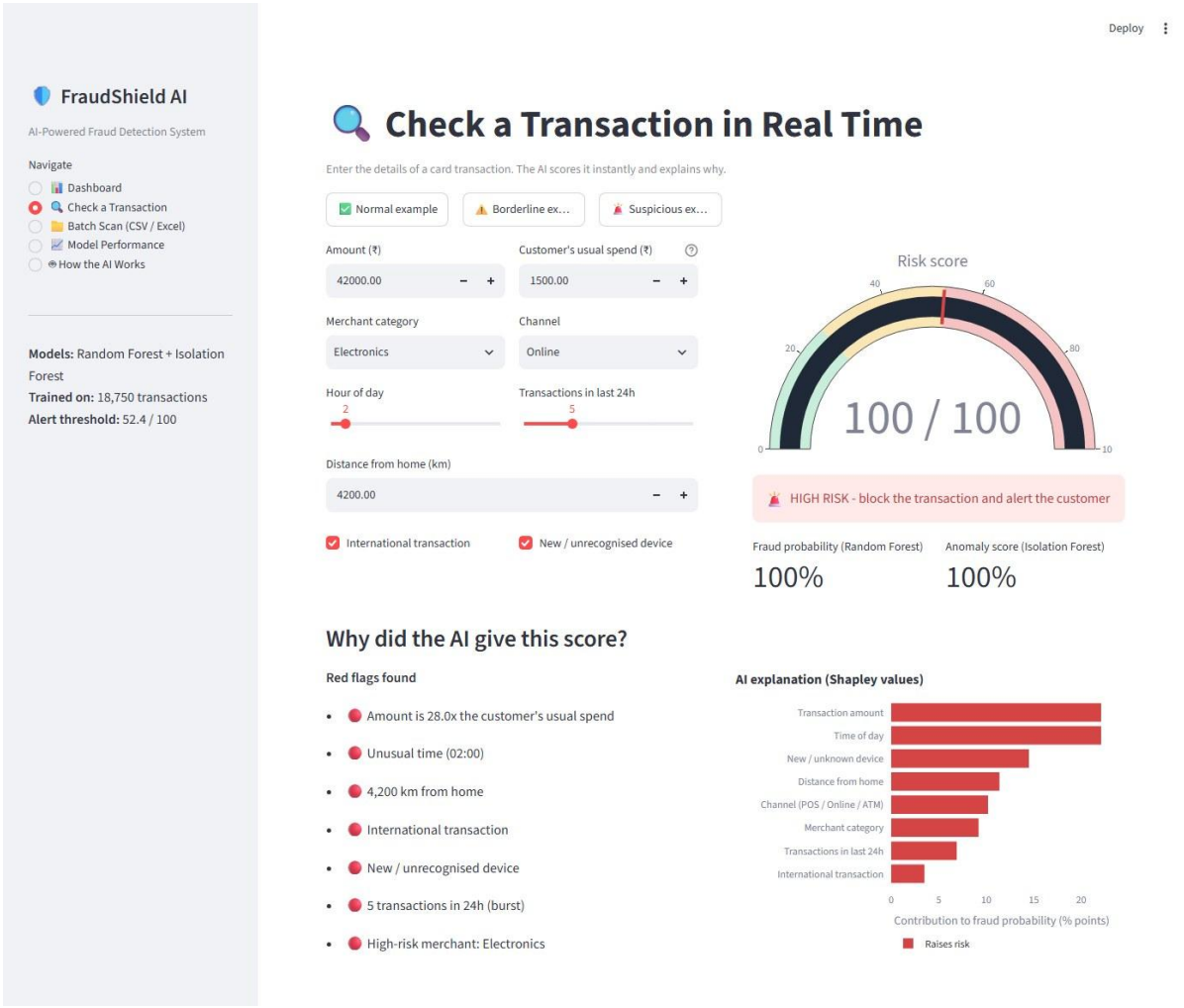


Figure 9 – Suspicious transaction: high risk, block – with red flags and Shapley explanation

Test 2 – 10-customer Excel file (Batch Scan page)

File `data/test_10_customers.xlsx` contains 10 customers with a mix of normal and suspicious transactions. Scanning it on the Batch Scan page gives:

Customer	Transaction	Risk score	Level
Ananya Rao	₹1,850 groceries, 11 am	0.1	Low
Rahul Mehta	₹950 restaurant, 8 pm	1.7	Low
Priya Nair	₹3,200 fuel, 9 am	2.8	Low
Vikram Singh	₹5,400 clothing online, 3 pm	6.1	Low
Kavya Reddy	₹2,300 utilities online, 10 am	5.5	Low
Sneha Iyer	₹9,500 travel (3.2× usual), 650 km away	29.8	Medium
Arjun Kapoor	₹5,200 electronics (3.3× usual), new device	41.6	Medium
Rohit Sharma	₹18,500 gift cards, 3 am, new device, 1,200 km	100	High
Meera Joshi	₹46,000 electronics, 1 am, abroad, 5 txns in 24 h	100	High
Karan Patel	₹25,000 crypto, 2 am, new device, 4 txns in 24 h	100	High

Result: 5 Low, 2 Medium, 3 High – exactly as expected; ₹89,500 at risk would be blocked.

Test 3 – 1,000 brand-new transactions

The built-in sample of 1,000 unseen transactions was scanned: **34 flagged as high risk, 25 medium**. Checking against the true labels, the system caught **28 of the 30 real frauds (93%)** and 82% of its alerts were correct.

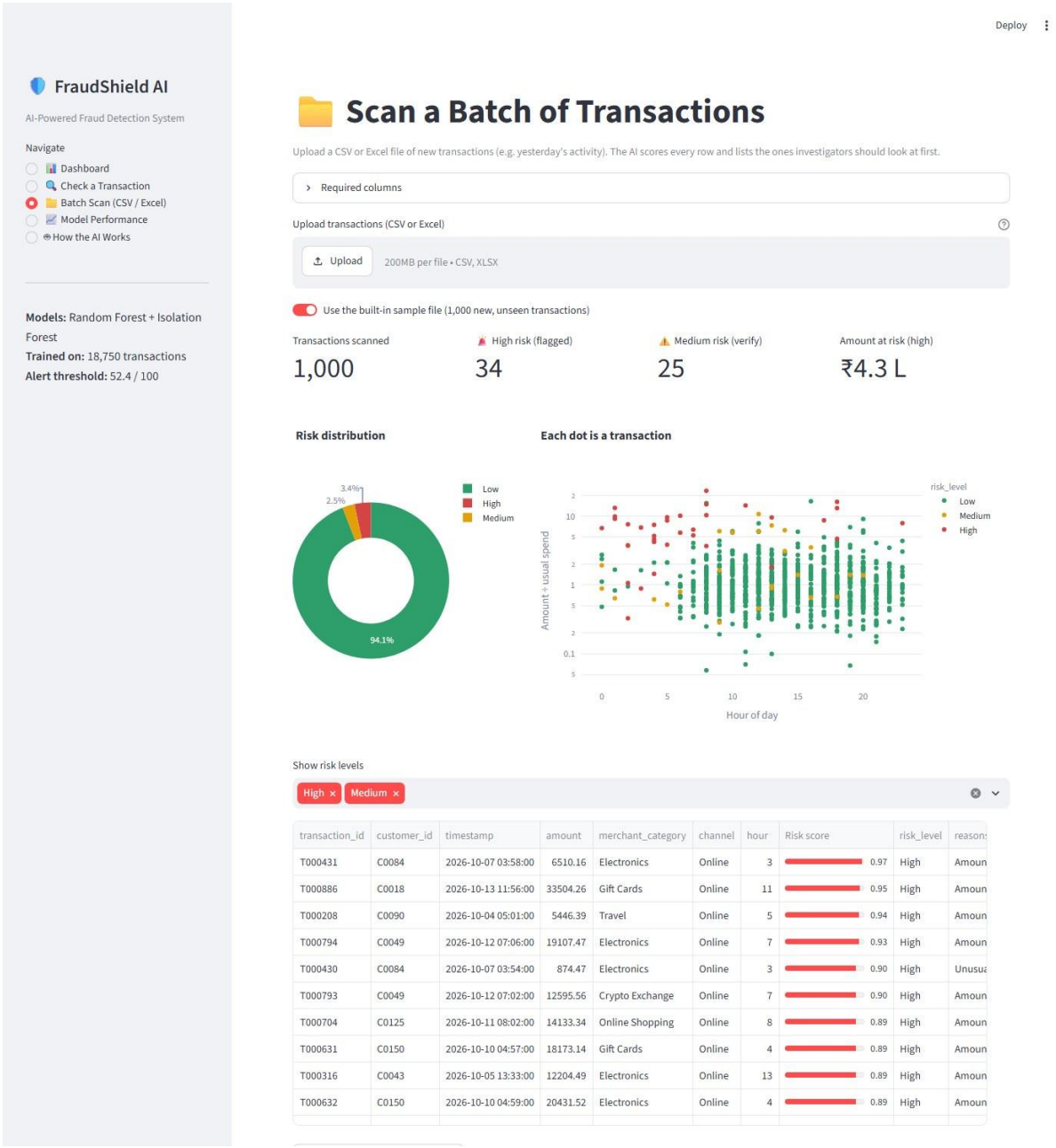


Figure 10 – Batch Scan page with the ranked list of risky transactions

> **✓ Output of this step**
> All features verified working with sample data.

Step 12 – Package and submit

1. Wrote the documentation: `README.md` (how to run) and this project report.
2. Created a private **GitHub repository** and pushed the code: https://github.com/Ramchandrappa/fraudshield-ai

3. Created `FraudShield_AI_Submission.zip` containing the source code, data, trained model and reports, and tested that it runs on its own.

File	Purpose
app.py	Streamlit web application (user interface)
fraud_model.py	AI core – features, hybrid model, training, evaluation, explanations
data_generator.py	Builds the synthetic transaction dataset
data/transactions.csv	25,000 labelled transactions
data/sample_new_transactions.csv	1,000 new transactions for the batch demo
data/test_10_customers.xlsx	10-customer Excel test file with expected results
models/	Saved trained model and evaluation metrics
requirements.txt, README.md	Packages needed; how to run

✓ Output of this step

Source code, report and test data ready for submission.

Part C – Step-by-Step User Guide

C1. Start the application

1. Install Python 3.10 or newer.
2. Open a terminal in the project folder and install the packages:

```
pip install -r requirements.txt
```

3. Start the app:

```
python -m streamlit run app.py
```

4. The browser opens http://localhost:8501. Use the left sidebar to move between pages. Press Ctrl + C in the terminal to stop.

C2. Check a single transaction

1. Open 🔍 **Check a Transaction**.
2. Click ✅ **Normal**, ⚠️ **Borderline** or 🚨 **Suspicious example** – or type your own values: amount, customer's usual spend, merchant, channel, hour, transactions in 24 h, distance from home, and tick International / New device if they apply.
3. Read the **risk gauge** and the decision: green = approve, yellow = verify, red = block.
4. Read **Red flags found** and the **Shapley chart** to see why.

C3. Scan a batch of transactions

1. Open 📁 **Batch Scan (CSV / Excel)**.
2. Keep "Use the built-in sample file" ON to scan 1,000 transactions, or switch it OFF and upload your own CSV / Excel file (e.g. `data/test_10_customers.xlsx`).
3. Review the summary cards, the risk pie chart and the scatter plot.
4. Use "Show risk levels" to filter; the table is sorted with the riskiest first and shows the reasons.
5. Click **Download scored results** to save a CSV.

Your file must contain these columns: `amount, customer_avg_amount, merchant_category, channel, hour, distance_from_home_km, is_foreign, is_new_device, txn_count_24h` (is_foreign and is_new_device are 0 or 1).

C4. Other pages

- 📊 **Dashboard** – view fraud patterns by hour, merchant, amount and red flag.
- 📈 **Model Performance** – confusion matrix, ROC curve, model comparison, feature importance.
- 🤖 **How the AI Works** – plain-language summary for presentations.

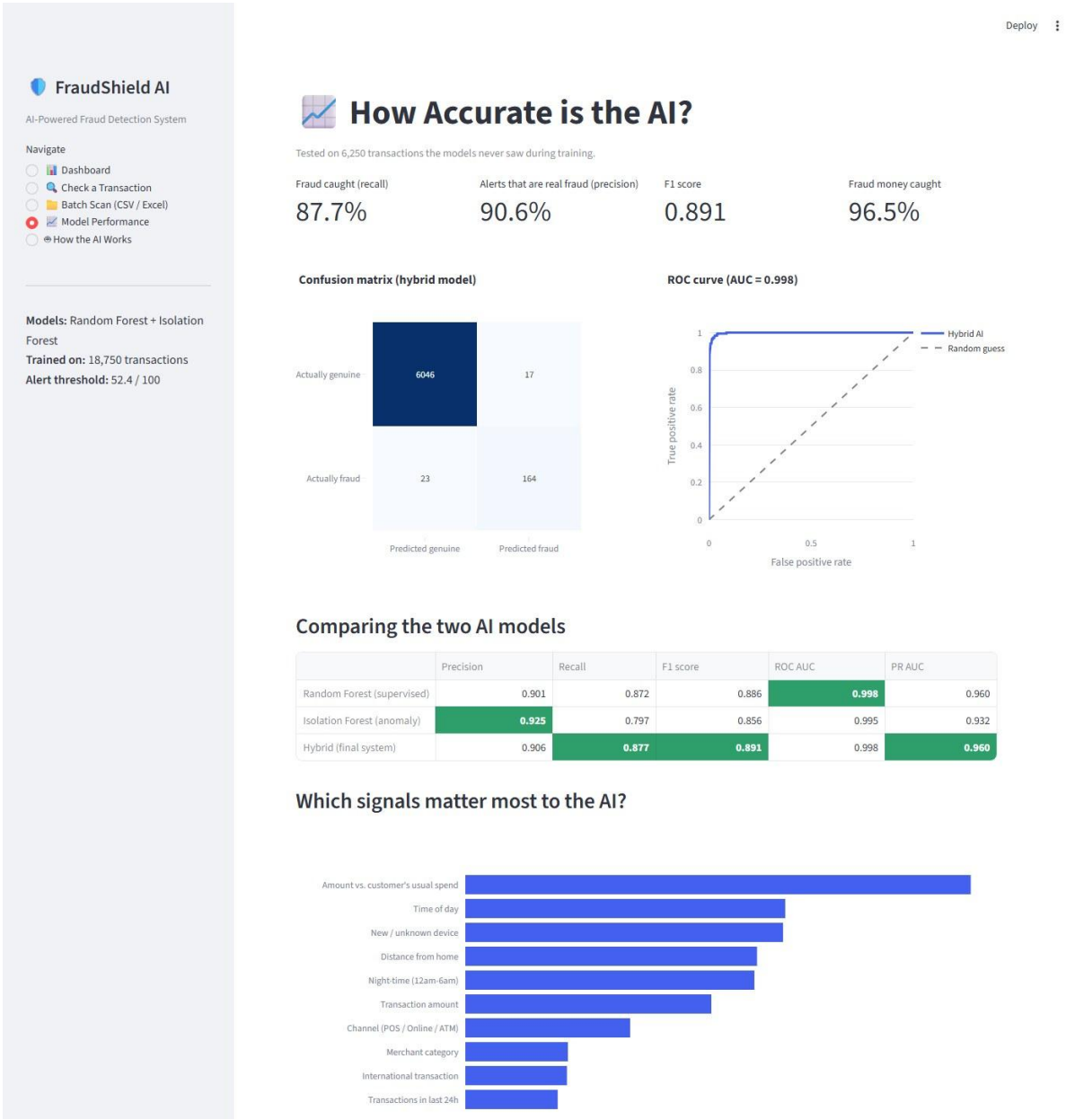


Figure 11 – Model Performance page

Part D – Applications, Limitations and Conclusion

D1. Benefits and practical applications

- **Banks and card networks** – score card payments in real time, before they are approved.
- **UPI, wallet and e-commerce platforms** – detect account takeover and payment fraud at checkout.
- **Insurance and corporate finance** – flag unusual claims or expense reimbursements for audit.
- **Benefits** – less money lost, fewer genuine customers blocked than with fixed rules, faster investigations (alerts are ranked and explained), easier compliance (every decision is explainable).

D2. Limitations

- The data is synthetic; real-world accuracy would be lower because real data is noisier and fraudsters actively try to look normal.
- The model is trained once; fraud patterns change, so it would need regular retraining.

D3. Future work

- Train on real, anonymised bank data and retrain regularly.
- Add richer signals – IP address, device fingerprint, merchant reputation, customer spending network.
- Try gradient boosting (XGBoost / LightGBM) and deep-learning sequence models.
- Learn from investigator feedback (active learning) and deploy as a real-time API.

D4. Conclusion

FraudShield AI shows how artificial intelligence can solve a real financial problem. By combining a supervised Random Forest with an unsupervised Isolation Forest, the system caught 87.7% of fraud and 96.5% of the fraud money, while wrongly flagging only 0.3% of genuine customers – and it explains every decision in plain language. The same approach is used by banks and payment companies to protect customers in real time.

Appendix A – How AI tools were used to build the project

Development activity	How the AI assistant helped
Planning	Suggested the hybrid Random Forest + Isolation Forest design and the app's page structure
Data	Wrote the synthetic data generator using documented real-world fraud patterns
Modelling	Wrote the training, threshold tuning and evaluation code; ensured tuning used validation data only
Explainability	Implemented red-flag reasons and exact Shapley values; fixed an explanation bug found during testing
User interface	Built the Streamlit app, then tested every page in a browser and fixed layout and readability issues
Documentation	Drafted the README, reports, test files and presentation notes